Practical 7

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

Write a program to :

- Calculate current need of resources by each process. Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

Process	Allocation				MAX			
	Printe	ROM	Hard Disk	RAM	Printe	ROM	Hard Disk	RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	o	1	2
Excel	1	o	0	0	.1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	ROM	Hard Disk	RAM
1	6	2	0

CODE and OUTPUT:

MINGW64:/c/Users/VICTUS/Desktop

GNU nano 8.7 banker.c Modified

```c
#include <stdio.h>

int main() {

    int i,j,k;

    int process = 5;
    int resource = 4;

    int allocation[5][4] = {
        {0,0,1,4},
        {0,6,3,2},
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4}
    };

    int max[5][4] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[4] = {1,6,2,0};

    int need[5][4];
    int finish[5] = {0};
    int safeSeq[5];
    int work[4];

    for(i=0;i<resource;i++)
        work[i] = available[i];

    printf("Need Matrix:\n");

    for(i=0;i<process;i++){
        for(j=0;j<resource;j++){
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }

    int count = 0;

    while(count < process){
        int found = 0;

        for(i=0;i<process;i++){

            if(finish[i]==0){
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-B Previous ^← Back ^Prev Word
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^B Where Was M-F Next ^→ Forward ^Next Word

GNU nano 8.7 banker.c

```c
    printf("Need Matrix:\n");

    for(i=0;i<process;i++){
        for(j=0;j<resource;j++){
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }

    int count = 0;

    while(count < process){
        int found = 0;

        for(i=0;i<process;i++){

            if(finish[i]==0){

                int flag = 0;

                for(j=0;j<resource;j++){
                    if(need[i][j] > work[j]){
                        flag = 1;
                        break;
                    }
                }

                if(flag==0){

                    for(k=0;k<resource;k++)
                        work[k] += allocation[i][k];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found==0){
            printf("\nSystem is NOT SAFE\n");
            return 0;
        }
    }

    printf("\nSystem is SAFE\nSafe Sequence:\n");

    for(i=0;i<process;i++)
        printf("P%d ",safeSeq[i]);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-B Pre

MINGW64:/c/Users/VICTUS/Desktop

```
VICTUS@AAM MINGW64 ~ (master)
$ cd Desktop

VICTUS@AAM MINGW64 ~/Desktop (master)
$ nano banker.c

VICTUS@AAM MINGW64 ~/Desktop (master)
$ gcc banker.c -o banker

VICTUS@AAM MINGW64 ~/Desktop (master)
$ ./banker
Need Matrix:
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

System is SAFE
Safe Sequence:
P1 P2 P3 P4 P0
VICTUS@AAM MINGW64 ~/Desktop (master)
$ |
```